Bancos de dados

Escolher pelo problema, não pela ferramenta

Modelos, propósitos, garantias e desempenho.

Uma loja virtual. Várias perguntas. Diferentes necessidades.

Nosso desafio: a loja virtual

Necessidade	Pergunta do negócio
Pedidos e estoque	Podemos confirmar esta compra?
Catálogo e busca	Qual produto atende à necessidade do cliente?
Eventos e medições	O que aconteceu e como os valores evoluíram?
Relatórios e conexões	Como estão as vendas? Há relações suspeitas?

> O mesmo domínio pode exigir formas diferentes de consultar os dados.

Por que não salvar tudo em JSON?

Imagine **uma unidade em estoque** e duas compras simultâneas.

- Quem coordena as alterações feitas ao mesmo tempo?
- O que acontece se o processo cair no meio da gravação?
- Como localizar pedidos sem ler e processar todo o arquivo?

Um gerenciador de banco oferece mecanismos de **consulta, concorrência, integridade e recuperação**.

> Persistir é apenas parte do problema.

Fontes: PostgreSQL · transações · PostgreSQL · constraints · PostgreSQL · backup e restauração

Transações: o significado de ACID

Propriedade	O que precisamos garantir
Atomicidade	Confirmar o conjunto de alterações ou desfazê-lo.
Consistência	Preservar as regras e invariantes definidos.
Isolamento	Controlar a interação entre transações concorrentes.
Durabilidade	Preservar o que foi confirmado, conforme as garantias configuradas.

> Uma transação não corrige, sozinha, toda condição de corrida.

Fontes: PostgreSQL · transações · PostgreSQL · isolamento

Banco não significa servidor separado

Opção	Cenário para avaliação
Arquivo de configuração	Poucos dados, leitura simples, alterações controladas.
SQLite embarcado	Aplicação local que precisa de SQL e transações.
Banco cliente-servidor	Aplicações e usuários compartilhando dados pela rede.

SQLite é um banco relacional, mesmo sem um serviço independente.

> Escolha a complexidade necessária, não a maior arquitetura possível.

Fontes: SQLite · quando usar

Tipos de banco

O modelo deve servir ao acesso

Para cada tipo, vamos perguntar:

Para que serve? Onde ajuda? Qual é o limite? Como otimizar?

Não existe apenas “SQL versus NoSQL”

Perspectiva	Exemplos de classificação
Modelo de dados	Relacional, documental, chave-valor, grafo.
Organização de armazenamento	Orientado a linhas ou a colunas.
Carga de trabalho	Transacional, analítica, temporal ou busca.

Um produto pode aparecer em **mais de uma categoria**.

PostgreSQL armazena JSON. ClickHouse utiliza SQL e armazenamento colunar.

Fontes: PostgreSQL · JSON · ClickHouse · armazenamento colunar

Relacional: entidades e regras

Exemplos: PostgreSQL, MySQL, SQL Server e SQLite.

```
clientes  →  pedidos  →  itens_pedido  ←  produtos
```

Propósito: relacionar entidades e proteger alterações com regras explícitas.

Na loja: clientes, pedidos, pagamentos registrados e estoque.

Ponto forte: transações e integridade referencial.
Cuidado: esquema bem definido não garante consultas eficientes.

Fontes: PostgreSQL · constraints · PostgreSQL · EXPLAIN

Relacional: combinar informações

“Quais são os últimos pedidos deste cliente?”

```sql
SELECT p.id, p.criado_em, c.nome
FROM pedidos AS p
JOIN clientes AS c ON c.id = p.cliente_id
WHERE p.cliente_id = 42
ORDER BY p.criado_em DESC, p.id DESC
LIMIT 20;
```

Otimização: investigar plano, volume processado e índices para os filtros e a ordenação.

> JOIN não é defeito. Trabalho desnecessário é o problema.

Fontes: PostgreSQL · EXPLAIN · PostgreSQL · índices compostos

Relacional: modelagem e otimização

Decisão	Raciocínio
Normalizar	Reduzir repetição de fatos e inconsistências.
Preservar um histórico	Guardar no item o preço praticado na compra.
Criar um índice composto	Atender uma consulta, não uma coluna isolada.

Um índice em `(cliente_id, criado_em)` **não equivale** a dois índices separados.

Índices também ocupam espaço e precisam ser mantidos nas gravações.

Fontes: Cassandra · modelagem · PostgreSQL · índices compostos · PostgreSQL · índices

Documental: um conjunto em um documento

Exemplo: MongoDB. **Na loja:** catálogo com atributos variáveis.

```
{
  "sku": "NOTE-001",
  "categoria": "informatica",
  "preco_centavos": 450000,
  "atributos": {
    "memoria_gb": 16,
    "armazenamento_gb": 512
  }
}
```

Ponto forte: reunir dados acessados juntos em uma estrutura flexível.

Fontes: MongoDB · modelagem

Documental: incorporar ou referenciar?

Incorpore dados lidos juntos e com crescimento controlado.
Referencie dados independentes, compartilhados ou que crescem sem limite.

```
db.produtos.createIndex({ categoria: 1, preco_centavos: 1 });
db.produtos.find({ categoria: "informatica" })
  .sort({ preco_centavos: 1 }).limit(20);
```

Otimização: evite documentos gigantes; combine consulta e índice.

> Esquema flexível não significa ausência de modelagem.

Fontes: MongoDB · modelagem · MongoDB · índices e ESR

Chave-valor: acesso por uma chave conhecida

Exemplo: Redis.

```
produto:42:resumo  →  {"nome":"Mochila"}
```

Propósito: recuperar e atualizar valores por identificadores diretos.

Na loja: cache, contadores e rankings.

Redis também oferece hashes, conjuntos e conjuntos ordenados.

Limite: consultas com muitas relações exigem outras estruturas ou recursos.

Fontes: Redis · tipos de dados

Redis: desempenho exige política de dados

- Para cache, defina **expiração, limite de memória e eviction**.
- Use a estrutura certa e acompanhe acertos e falhas do cache.
- Avalie **pipelining** para reduzir viagens de rede.

Pipelining não é uma transação atômica.

Redis oferece persistência, mas cache descartável e dado durável exigem decisões diferentes.

> Se este cache desaparecer, conseguimos reconstruí-lo?

Fontes: Redis · eviction · Redis · pipelining · Redis · persistência

Wide-column: a consulta orienta o modelo

Exemplo: Apache Cassandra.

Propósito: distribuir dados e atender padrões de acesso conhecidos.

Na loja: eventos de entrega por transportadora e período.

Trecho ilustrativo de chave primária em CQL:

```
PRIMARY KEY ((transportadora_id, dia), ocorrido_em, evento_id)
```

A **chave de partição** distribui os dados; as demais colunas da chave organizam os registros na partição.

Fontes: Cassandra · modelagem

Wide-column: limites e otimizações

Otimização: distribuir a carga, limitar o tamanho das partições e consultar poucas partições.

Cuidado: no Cassandra, não há joins relacionais; diferentes consultas podem exigir duplicação de dados.

Wide-column	Colunar analítico
Exemplo: Cassandra.	Exemplo: ClickHouse.
Acesso orientado por chaves e partições.	Leitura de colunas para grandes análises.

> O nome parecido não significa o mesmo propósito.

Fontes: Cassandra · modelagem · ClickHouse · armazenamento colunar

Grafos: as conexões são parte da resposta

Exemplo: Neo4j.

```
(Cliente A) ———USA———> (Dispositivo) <———USA——— (Cliente B)
```

Propósito: representar entidades e percorrer seus relacionamentos.

Na loja: investigar contas que compartilham dispositivos ou endereços.

Ponto forte: responder perguntas sobre caminhos e conexões.

> Ter relacionamentos não basta para justificar um banco de grafos.

Fontes: Neo4j · conceitos

Grafos: limite o trabalho da travessia

```
MATCH (c:Cliente {id: $id})-[:USA]->(d:Dispositivo)
      <-[:USA]-(outro:Cliente)
WHERE outro <> c
RETURN DISTINCT outro.id
LIMIT 20;
```

Otimização: indexe o ponto de partida, use relações específicas e limite caminhos e resultados.

Cuidado: uma travessia sem limites pode explorar uma parte enorme do grafo.

Fontes: Neo4j · otimização de consultas

Séries temporais: valores ao longo do tempo

Exemplo: InfluxDB.

Instante	Equipamento	Temperatura
10:00	câmara-01	3,2 °C
10:01	câmara-01	3,4 °C
10:02	câmara-01	3,3 °C

Na loja: acompanhar equipamentos do centro de distribuição.

Pergunta: como a temperatura evoluiu em cada intervalo?

Modelo: instante + identificação da série + valores medidos.

Fontes: InfluxDB · modelagem

Séries temporais: retenção e agregação

- Consulte **janelas de tempo**, não todo o histórico por padrão.
- Separe identificação da série e valores medidos.
- Avalie escrita em lote e a precisão temporal necessária.

Decisão de projeto: manter medições detalhadas recentes e resumos antigos pode reduzir volume e trabalho.

Cuidado: recomendações de modelagem variam entre motores e versões.

Fontes: InfluxDB · modelagem · InfluxDB · otimização de escrita

Colunar analítico: analisar muitos registros

Exemplo: ClickHouse.

OLTP — operacional	OLAP — analítico
Confirmar uma compra.	Somar vendas de todo o histórico.
Alterar poucos registros.	Agregar grandes conjuntos.
Operações do dia a dia.	Indicadores e exploração de dados.

Armazenamento colunar favorece a leitura das **colunas necessárias** à análise.

> SQL é uma linguagem. OLTP e OLAP descrevem cargas de trabalho.

Fontes: ClickHouse · armazenamento colunar

Colunar: leia menos, grave em lotes

```sql
SELECT categoria, SUM(valor_centavos) AS receita
FROM vendas_analiticas
WHERE data >= '2025-01-01'
GROUP BY categoria;
```

Otimização no ClickHouse: alinhar a chave de ordenação aos filtros, selecionar as colunas necessárias e agrupar inserções.

Cuidado: otimizar grandes análises não significa substituir automaticamente o banco que confirma pedidos.

Fontes: ClickHouse · chave de ordenação · ClickHouse · estratégias de inserção

Motores de busca: texto e relevância

Exemplo: Elasticsearch.

Propósito: encontrar e ordenar resultados a partir de conteúdo textual e critérios de pesquisa.

Na loja: procurar “mochila para notebook” no catálogo.

Busca exata	Busca textual
Encontrar o SKU `MOCH-42`.	Encontrar descrições relevantes.
Comparar um identificador.	Analisar termos e calcular relevância.

Cuidado: uma alteração pode não aparecer imediatamente na busca.

Fontes: Elasticsearch · campos text · Elasticsearch · busca quase em tempo real

Busca: modelagem também é otimização

```
{
  "properties": {
    "nome": { "type": "text" },
    "categoria": { "type": "keyword" }
  }
}
```

Otimização: use tipos adequados, indexação em lote e uma frequência de refresh compatível com a necessidade de atualização.

Decisão da loja: o índice pode ser reconstruído a partir da fonte principal.

Fontes: Elasticsearch · campos text · Elasticsearch · indexação

Vetorial: buscar por proximidade

Exemplo: PostgreSQL com a extensão pgvector.

```
Texto ou imagem → modelo de embeddings → vetor → busca
```

Propósito: localizar representações numericamente semelhantes.

Na loja: recomendar itens próximos ao significado de uma descrição.

Ponto forte: busca por similaridade além da igualdade de palavras.

Limite: similaridade não comprova verdade nem relevância para o negócio.

Fontes: pgvector · documentação

Vetorial: velocidade versus recall

```
SELECT id, nome
FROM produtos_vetoriais
ORDER BY embedding <=> '[0.2,0.1,0.9]'::vector
LIMIT 5;
```

Vetor reduzido e ilustrativo; `<=>` representa distância de cosseno.

Otimização: comparar busca exata e índices aproximados, como HNSW e IVFFlat, com filtros reais.

Meça: latência, memória e **recall** — quanto do resultado exato foi recuperado.

Fontes: pgvector · documentação

Comparação: qual pergunta cada tipo favorece?

Tipo	Pergunta característica	Exemplo
Relacional	Quais entidades e regras participam da operação?	PostgreSQL
Documental	Quais informações são acessadas juntas?	MongoDB
Chave-valor	Qual valor está nesta chave?	Redis
Wide-column	Quais registros pertencem a esta partição?	Cassandra
Grafo	Como estas entidades estão conectadas?	Neo4j
Séries temporais	Como este valor evoluiu no período?	InfluxDB
Colunar analítico	Qual resultado de agregar muitos registros?	ClickHouse
Busca textual	Quais resultados são relevantes para este texto?	Elasticsearch
Vetorial	Quais representações são mais próximas?	pgvector

Como escolher

Requisitos antes de produtos

Consultas + garantias + carga + operação

A melhor ferramenta para outro projeto pode ser a errada para este.

Cinco perguntas antes de decidir

Pergunta	O que precisamos conhecer
Como os dados serão acessados?	Consultas, filtros, ordenações e relações.
Quais regras não podem ser violadas?	Transações e garantias de leitura e escrita.
Qual é a carga?	Volume, crescimento, concorrência e picos.
O que acontece em uma falha?	Indisponibilidade, atraso e perda aceitáveis.
Quem sustenta a operação?	Equipe, manutenção, recuperação e custo.

> Valide com uma carga representativa, não com slogans.

Uma evolução possível para a loja

Ponto de partida para avaliação: PostgreSQL no núcleo transacional.

Só acrescentar quando…	Candidato
Houver leituras repetidas que possam ser reaproveitadas.	Cache com Redis.
A busca exigir recursos específicos de relevância.	Elasticsearch.
A análise pesada justificar uma carga separada.	ClickHouse.

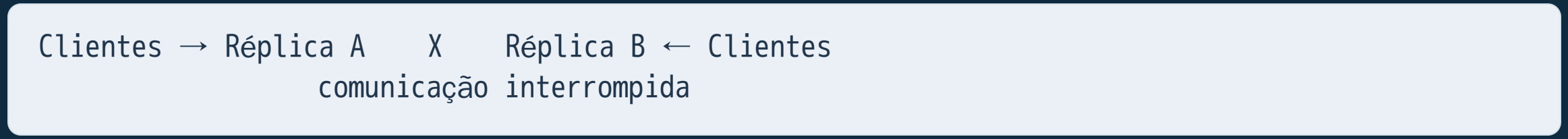
Um catálogo flexível também pode usar **JSONB no PostgreSQL**.

> Cada banco adicional precisa pagar seu custo de integração e operação.

Fontes: PostgreSQL · JSON

Teorema CAP

E quando as réplicas deixam de se comunicar?



As duas réplicas continuam funcionando, mas estão **isoladas entre si**.

CAP: três conceitos específicos

Letra	Significado no teorema
C — Consistência	Linearizabilidade: operações se comportam como uma única cópia, respeitando a ordem real.
A — Disponibilidade	Requisições a nós não falhos terminam com uma resposta válida para a operação.
P — Partição de rede	A comunicação entre partes do sistema pode ser interrompida.

A não significa apenas uptime, resposta rápida ou devolver um erro.

Fontes: Gilbert e Lynch · Perspectives on CAP

Uma leitura durante a partição

Cenário hipotético: o preço inicial era **R$ 100** nas duas réplicas.

Passo	Réplica A	Réplica B
A comunicação é interrompida.	Mantém R$ 100.	Mantém R$ 100.
Uma atualização é confirmada em A.	Passa a R$ 120.	Não recebe a alteração.
Depois, chega uma leitura em B.	Continua isolada.	Qual preço deve retornar?

B não tem como confirmar se o valor local continua atualizado.

> Ela não consegue inventar a informação que não recebeu.

Durante a partição, há uma escolha

Preservar a consistência	Preservar a disponibilidade
Não devolver um estado que pode estar desatualizado.	Responder com os dados disponíveis localmente.
Impedir a conclusão de certas operações.	Aceitar que algumas respostas possam divergir.

CAP: não é possível garantir simultaneamente C e A sob partições de rede.

Na loja, podemos discutir atraso em recomendações e exigir garantias diferentes para operações críticas.

Fontes: Eric Brewer · CAP Twelve Years Later

CAP: o que não devemos concluir

“Escolha quaisquer duas” é uma simplificação ruim: a tensão entre C e A aparece diante da partição.

“Este produto é sempre CP ou AP” esconde configurações e garantias por operação.

C do ACID ≠ C do CAP: preservar invariantes não é o mesmo que linearizabilidade.

> Pergunte qual comportamento a operação oferece durante a falha.

Fontes: Eric Brewer · CAP Twelve Years Later · PostgreSQL · isolamento

Boas práticas e desempenho

Otimizar é evitar trabalho desnecessário

Medir → entender → alterar → medir novamente.

Trocar de banco não deve ser a primeira resposta a uma consulta lenta.

Modele regras, não apenas campos

```sql
CREATE TABLE produtos (
  id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  sku TEXT NOT NULL UNIQUE,
  nome TEXT NOT NULL,
  estoque INTEGER NOT NULL CHECK (estoque >= 0)
);
```

`NOT NULL`, `UNIQUE`, `CHECK` e chaves estrangeiras protegem regras expressáveis no banco.

Consultar antes de inserir não substitui uma restrição de unicidade.

Fontes: PostgreSQL · constraints

Concorrência: verificar e alterar juntos

Exemplo PostgreSQL para reservar uma unidade:

```
UPDATE produtos
SET estoque = estoque - 1
WHERE id = 42 AND estoque >= 1
RETURNING id, estoque;
```

Nenhuma linha retornada: a reserva não foi realizada.

Para combinar reserva e pedido, use uma transação e trate falhas e repetições da operação.

> Apenas ler o estoque antes não impede uma disputa concorrente.

Fontes: PostgreSQL · isolamento · PostgreSQL · transações

Segurança: dado não é comando

Trecho com `node-postgres`; `pool` já está configurado:

```
const resultado = await pool.query(
  'SELECT id, nome FROM clientes WHERE email = $1',
  [email]
);
```

Use parâmetros, não concatenação de entradas no SQL.

A aplicação deve ter permissões mínimas; credenciais e dados sensíveis não devem aparecer no código ou nos logs.

Fontes: OWASP · prevenção de SQL injection · node-postgres · consultas parametrizadas

ORM: observe o que realmente é executado

Problema N+1:

```
1 consulta busca 100 pedidos.
100 consultas adicionais buscam seus clientes.
Total: 101 consultas.
```

Possíveis correções: projeções, joins ou carregamento em lote.

Retorne os campos necessários e limite o resultado.

> Trocar tudo para eager pode apenas mudar o problema de lugar.

Fontes: Hibernate · carregamento de associações

Antes de otimizar, procure evidências

Evidência	Pergunta
Frequência e latência	Qual consulta mais pesa para o usuário?
Plano de execução	Como o banco encontra, combina e ordena dados?
Linhas e buffers	Quanto trabalho foi feito para retornar o resultado?
Esperas e recursos	Há disputa, conexões demais ou limite de capacidade?

Hipótese primeiro: “a consulta está lendo muitos registros para devolver poucos”.

Depois, escolha uma alteração e compare novamente.

Fontes: PostgreSQL · EXPLAIN

Demonstração: investigue antes de alterar

Base de estudo: **200 mil pedidos fictícios**, com 1.000 clientes.

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT id, criado_em, total_centavos
FROM aula_db.pedidos_demo
WHERE cliente_id = 42
ORDER BY criado_em DESC, id DESC
LIMIT 20;
```

Qual caminho o banco escolheu? Precisou ordenar? Quanto trabalho realizou?

Atenção: `EXPLAIN ANALYZE` executa a consulta.

Fontes: PostgreSQL · EXPLAIN

Demonstração: alinhe índice e consulta

```
CREATE INDEX idx_pedidos_demo_cliente_data
ON aula_db.pedidos_demo
  (cliente_id, criado_em DESC, id DESC);
```

A ordem acompanha o acesso desejado:

cliente específico → pedidos mais recentes → desempate por id.

Execute novamente a mesma consulta e compare o plano.

> Não mudamos de banco. Mudamos a forma de encontrar os dados.

Fontes: PostgreSQL · índices compostos

Compare trabalho, não um número isolado

Antes e depois	O que observar
Acesso aos dados	Varredura sequencial ou acesso por índice?
Ordenação	Existe um passo separado de sort?
Processamento	Quantos registros e buffers participaram?
Resultado	As mesmas linhas e a mesma ordenação?

Cache, estatísticas, distribuição dos dados e concorrência influenciam a medição.

Seq Scan não é automaticamente ruim. Índice não é automaticamente bom.

Fontes: PostgreSQL · EXPLAIN · PostgreSQL · índices

Paginação: evite descartar demais

Um `OFFSET` grande ainda exige processar as linhas ignoradas.

Alternativa ilustrativa: cursor com data e id da última linha recebida.

```
SELECT id, criado_em, total_centavos
FROM aula_db.pedidos_demo
WHERE cliente_id = $1
  AND (criado_em, id) < ($2::timestamptz, $3::bigint)
ORDER BY criado_em DESC, id DESC
LIMIT 20;
```

Cuidado: cursor precisa de ordem estável e não oferece salto livre para qualquer página.

Fontes: PostgreSQL · LIMIT e OFFSET · PostgreSQL · índices compostos

Operação faz parte da qualidade

Recuperação: defina a perda de dados tolerável e o prazo para restabelecer o serviço; teste a restauração.

Replicação não substitui backup: uma exclusão indevida também pode chegar à réplica.

Evolução: versione migrações e planeje compatibilidade entre o código e o esquema.

Observabilidade: acompanhe falhas, consultas lentas e crescimento da base.

> Ter uma cópia não é o mesmo que comprovar a recuperação.

Fontes: PostgreSQL · backup e restauração

Checklist antes de adicionar complexidade

A consulta está clara? Sabemos quais dados precisamos retornar?

O modelo e o acesso combinam? Índices, partições e estruturas fazem sentido?

As garantias estão protegidas? Concorrência, validação e recuperação foram tratadas?

Existe evidência do limite? Medimos uma carga representativa?

> Depois disso, avalie cache, réplicas, distribuição ou outro motor.

Discussão: escolha e justifique

Cenário	O que precisa funcionar bem?
Pedido e reserva de estoque.	Alterações relacionadas sem violar a regra.
Catálogo com atributos diferentes.	Consultar e atualizar as estruturas do catálogo.
Cache do resumo dos produtos.	Recuperar por chave e reconstruir quando necessário.
Contas que compartilham dispositivos.	Percorrer conexões entre entidades.
Análise do histórico de vendas.	Agregar grandes conjuntos de registros.

Indique um candidato, uma vantagem e um custo da escolha.

Respostas possíveis, não obrigatórias

Cenário	Candidato inicial	Cuidado
Pedido e estoque.	Relacional.	Transação e concorrência bem implementadas.
Catálogo variável.	Documental ou relacional com JSON.	Limites dos documentos e consultas reais.
Cache de produtos.	Chave-valor.	Expiração, invalidação e reconstrução.
Conexões entre contas.	Grafo.	Travessias e interpretação dos resultados.
Grandes análises.	Colunar analítico.	Atualização da cópia e custo operacional.

> A justificativa é mais importante do que acertar uma marca.

A decisão que importa

Escolha pelo padrão de acesso.
Proteja as regras e entenda as falhas.
Otimize com evidências.

O melhor banco atende às necessidades com uma complexidade que a equipe consegue sustentar.

Referências: fundamentos e implementação

Tema	Documentação e leitura
Transações e concorrência	PostgreSQL: transações · Isolamento
Integridade e índices	Constraints · Índices compostos
Diagnóstico e recuperação	EXPLAIN · Backup
Segurança e acesso	OWASP: SQL injection · node-postgres
CAP	Gilbert e Lynch: Perspectives on CAP · Eric Brewer: CAP Twelve Years Later

Fontes técnicas oficiais e textos dos autores do teorema.

Referências: modelos e especializações

Tema	Documentação
Relacional e JSON	SQLite · PostgreSQL JSON
Documental	MongoDB: modelagem · Diretriz ESR
Chave-valor	Redis: tipos · Eviction
Wide-column e grafos	Cassandra: modelagem · Neo4j: conceitos
Temporal e analítico	InfluxDB: modelagem · ClickHouse: introdução
Busca textual e vetorial	Elasticsearch: refresh · pgvector